

Documentオブジェクト

前回、**window** オブジェクト はブラウザ全体の「社長さん」だと説明しましたね。今回の **document** オブジェクト は、**window** オブジェクトの下にあるプロパティで、その社長さんの下で働く「現場監督（げんばかんとく）」です。

[!IMPORTANT]

オブジェクトは別のオブジェクトのプロパティになることがあります。

学生の方は、「ウェブページという建物を管理し、中身を書き換えたり、新しくパーツを作ったりする責任者」と考えると、イメージが湧きやすくなるかもしれません。

documentオブジェクトとは、ブラウザに表示されている**HTML（ウェブページの中身）** そのものを指します。JavaScriptを使って、文字の色を変えたり、ボタンが押されたときに動きをつけたりするのは、すべてこの「現場監督（document）」に命令を出しているからです。

これを専門用語で **DOM（ドム / Document Object Model）** と呼びます。HTMLを「木の枝」のような構造として管理しているのが特徴です。

DOMツリーについては、次のセクションで、くわしく説明します。ここでは、Windowとdocumentの違いをしっかりと頭に入れてください。

1. documentオブジェクトとは？

ブラウザに表示されている**HTML（ウェブページの中身）** そのものを指します。JavaScriptを使って、文字の色を変えたり、ボタンが押されたときに動きをつけたりするのは、すべてこの「現場監督（document）」に命令を出しているからです。

これを専門用語で **DOM（ドム / Document Object Model）** と呼びます。HTMLを「木の枝」のような構造として管理しているのが特徴です。

2. よく使うプロパティ（ページの情報）

現場監督が知っている、今のページの情報です。

- **document.title** ブラウザのタブに表示されている「ページのタイトル」です。JavaScriptで書き換えることもできます。
- **document.body** HTMLの `<body>` タグの中身全体です。ページ全体の背景色を変えたいときなどに使います。
- **document.URL** 現在開いているページの住所（URL）を教えてください。

3. よく使うメソッド（現場監督への命令）

これが一番大切です！ 特定の場所を探したり、中身を変えたりする命令です。

場所を探す（セレクタ）

- `getElementById("ID名")` 特定のIDがついたパーツを1つだけ探します。一番速くて正確です。
- `querySelector("セクタ")` CSSと同じ書き方（`.class` や `#id`）でパーツを探せます。とても便利です！

これらは、以前にも出てきましたね。実はdocumentオブジェクトのメソッドだったのです！

[!note]

`document.getElementById("ID名")` のように書いてもかまいませんが、普通は`document.`を省略して、`getElementById("ID名")`と書きます。

中身を書き換える・作る

- `createElement("タグ名")` 新しいパーツ（`<div>` や `<li>` など）をメモリの中に作ります。
- `write("文字")` ページに直接文字を書き込みます（※あまり使いませんが、基本として覚えておきましょう）。

4. 実際に動かしてみよう！

Visual Studio Code を使って、以下のようなhtmlを作成しましょう。名前は`doc_obj.html`としましょうか。

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <h1 id="greeting">こんにちは！</h1>
  <br>
  <p>コンソールを開いて、下のJavaScriptを入力してみましょう。</p>
  <br>
  <pre>
    // 1. IDが "greeting" というパーツを探す
    const message = document.getElementById("greeting");

    // 2. その中身を書き換える
    message.textContent = "こんにちは、日本へようこそ！";

    // 3. スタイル（色）も変えちゃう
    message.style.color = "red";
  </pre>
  <br>
  <a href="javascript:history.back();">戻る</a>
</body>
</html>
```

保存できたら、Visual Studio Code 上から、`Open with Live Server`を使って、開きましょう。

ブラウザに表示された指示に従って、コンソールにJavaScriptを入力しましょう。

コンソールを開くには、F12 キー（ノートPCは Fn + F12） または Ctrl + Shift + I を使います。

オブジェクトと `const` の意外な関係

上のコードを見て、疑問に思いませんでしたか？「`const` で宣言された変数は、後から値を変更できないはずでは？」 そう思った人は、変数の説明をとてもよく覚えている人です。

しかし、上のコードはエラーになりません。それについて、説明しましょう。

JavaScriptの初心者が混乱しやすい点のひとつが、オブジェクトと `const` の意外な関係です。

「`const` で宣言した変数は中身を変えられない」と説明しましたが、オブジェクトの場合は少し違います。

- 名札の固定（参照の不変性）：`const` で作ったオブジェクトそのものを別のオブジェクトに入れ替えることはできません（別の箱に名札を付け替えるのは禁止）。
- 中身の変更はOK：箱の中に入っている個別のデータ（プロパティ）を書き換えることは可能です。

「強力なガムテープで名札が貼られた箱」をイメージしてください。ガムテープで固定されているので、名札を別の新しい箱に貼り直すことはできません。でも、箱のふたをひとつ開けて、中に入っているお菓子を入れ替たり、ジャムを塗ったりすることは自由なのです。

例えるなら、「ファイルの保存場所（パス）」は固定されているけれど、「ファイルの中身」は自由に書き換えられるようなイメージです。

先ほどの例では、`const` を使って宣言した `message` という箱を開けて、中身を "こんにちは、日本へようこそ！" に入れ替えました。これはエラーにはなりません。

しかし、

```
// 1. IDが "greeting" というパーツを探す
const message = document.getElementById("greeting");

// 2. タグがpのパーツを探す
message = document.querySelector("p")
```

はエラーになります。`message` というラベルを「"greeting"というIDの箱」からはがして、「"p"というタグの別の箱」に貼ろうとしているからです。

`const` は `message` というラベルと、それを入れる入れ物との関係を固定して、ラベルの張替えを許しません。

しかし、あくまで **ラベルと入れ物との関係** であって、**入れ物が同じであれば、その中身が変わっても、かまわない** のです。

5. 学生へのアドバイス：`window` と `document` の使い分け

「どっちを使えばいいの？」と聞かれたら、こう答えてあげてください。

「ブラウザの機能（タイマー、アラート、URL移動）を使いたいときは **window**」

「ページの中身（文字、画像、ボタン）をいじりたいときは **document**」

まとめテーブル

命令・情報の名前	役割	例え話
title	ページのタイトル	本の表紙の名前
getElementById()	特定のパーツを指名する	「出席番号〇番の人！」と呼ぶ
querySelector()	自由にパーツを探す	「赤い服を着ている人！」と探す
textContent	文字を書き換える	看板の文字を書き直す
createElement()	新しい要素を作る	新しいレンガを用意する

いかがでしょうか？「現場監督の document くん」がいれば、ウェブページを思った通りに操れるようになります。

プロパティとメソッド

document オブジェクトは、ブラウザの画面《がめん》に映《うつ》っている「白い紙《かみ》（HTMLの文章《ぶんしょう》）」そのものです。これを使うことで、HTMLの文字《もじ》を読《よ》み取《と》ったり、新しく書き換《か》えたりすることができます。

留学生《りゅうがくせい》の方《かた》に分かりやすいよう、こちらもプロパティ（情報《じょうほう》）とメソッド（命令《めいれい》）に分けて整理《せいり》しました。

1. documentオブジェクトのプロパティ（情報《じょうほう》）

プロパティは、今見ているWebページの「中身や設定《せってい》のデータ」です。

📄 ページの中身（HTMLのパーツ）

- **body**（ボディ）：
- 意味《いみ》：HTMLの **<body>** タグの部分《ぶぶん》、つまり「ユーザーの目に見える画面全体」を指《さ》すオブジェクトです。ここに背景《はいけい》の色《いろ》を変える命令《めいれい》などを出せます。
- **forms**（フォームズ）：
- 意味《いみ》：ページの中にあるすべての **<form>** タグの「詰め合わせセット（リスト）」です。

[!note] 入力一覧にあった **forms** は、大文字小文字のタイポ（打ち間違い）になりやすいポイントです。正しくは **forms**（rのあとにo）だと教えてあげてください。

🌐 ネットワークやURLの情報《じょうほう》

- **URL** (ユーアールエル) :
- 意味《いみ》: 今見ているページの「完全なURL (アドレス)」の文字列です。
- **domain** (ドメイン) :
- 意味《いみ》: URLの中から、サーバーの「名前 (場所)」の部分だけを抜き出したものです (例《れい》: `google.com` や `yahoo.co.jp`)。
- **location** (ロケーション) :
- 意味《いみ》: `window.location` と同じ、URLの詳しい情報が入ったオブジェクトです。
- **referrer** (リファラー) :
- 意味《いみ》: ユーザーが「どこのページ (リンク元) からこのページに飛んできたか」という前のページのURLを教えてください。

🔥 ページの設定《せってい》・状態《じょうたい》

- **title** (タイトル) :
- 意味《いみ》: HTMLの `<title>` タグに書いた、ブラウザの「タブ」に出るページのタイトルです。JavaScriptから `document.title = "新しい名前";` と書けば、途中でタブの名前を変えられます。
- **lastModified** (ラスト・モディファイド) :
- 意味《いみ》: このHTMLファイルが「最後に更新《こうしん》(セーブ) されたのはいつか」という日時 (日付と時間) を教えてください。
- **cookie** (クッキー) :
- 意味《いみ》: ブラウザにユーザーのログイン状態《じょうたい》や設定《せってい》を一時的に保存《ほぞん》しておくための「小さなメモ帳」のようなデータです。

2. documentオブジェクトのメソッド (命令《めいれい》)

🔥 画面に文字を書く (現在はあまり使われません)

- **`**write("文字") / writeln("文字")**`** :
- 意味《いみ》: 画面に直接文字《ちょくせつもじ》を書き出す命令《めいれい》です。 `writeln` は最後に自動で改行《かいぎょう》(改行コード) を入れてくれます。

[!note] これらは初期のJavaScriptでよく使われましたが、現在は `document.createElement()` などの新しい書き方が主流《しゅりゅう》です。ページが完全に読み終わった後に `document.write()` を実行《じっこう》すると、**今ある画面の中身が全部消えて真っ白になってしまうという危険《きけん》**な罫《わな》があるため、留学生には「こういう古い歴史《れきし》の命令もあるよ」と紹介《しょうかい》する程度《ていど》が良いです。

🖨 文字を書くための「紙《かみ》」を開く・閉じる

- ****open() / close()**** :
- **意味《いみ》** : 上の `document.write()` で文字を書くために、「新しいドキュメントの紙を開く」命令《めいれい》と、書き終わったから「紙を閉じて画面を完成させる」命令《めいれい》です。

[!note] `window.open() / close()` は「ブラウザのウィンドウ（タブ）」を開き閉めしますが、`document.open() / close()` は「ページの中の文章（テキストデータ）」を開き閉めするので、ごっちゃんにしないよう注意《ちゅうい》が必要です。

3. 留学生《りゅうがくせい》向けのまとめ（青空文庫形式《あおぞらぶんこけいしき》）

****Webページを支配《しはい》する document**** `document` オブジェクトは、ブラウザに映《うつ》っているWebページの「白い紙《かみ》そのもの」です。

- **プロパティ（情報《じょうほう》）** : `document.title` を使えばタブの名前が分かり、`document.body` を使えば画面全体の色やスタイルを変えることができます。また、`forms` を使えばページの中にある入力ボックスを全部集めることができます。
- **メソッド（命令《めいれい》）** : `write()` などの画面に直接文字を書く古い命令もありますが、今のプログラミングではあまり使いません。

これからWebサイトの文字を書き換えたり、ボタンが押されたときの動きを作ったりするときは、必ずこの `document` からスタートします。JavaScriptで一番たくさん使う大黒柱《だいこくばしら》のようなオブジェクトなので、しっかり仲良《なかよ》くなりましょう！

いかがでしょうか。歴史《れきし》が長いオブジェクトなので古いメソッドも混《ま》ざっていますが、****body、forms、title、URL**** などのプロパティは今でも毎日《まいにち》のように使う主役《しゅやく》たちです。